What if we embraced simulation-driven development? 🤔

DEVOXX FRANCE 2025

Pierre Zemb clever cloud

$ whoami

- Pierre Zemb
- Staff Engineer clever cloud
- Working around distributed systems
  - Building, contributing, debugging, paging…
- Maintaining several OSS libraries in Rust
- Involved in local communities such as FinistDevs
- Squash player

$ (also) whoami

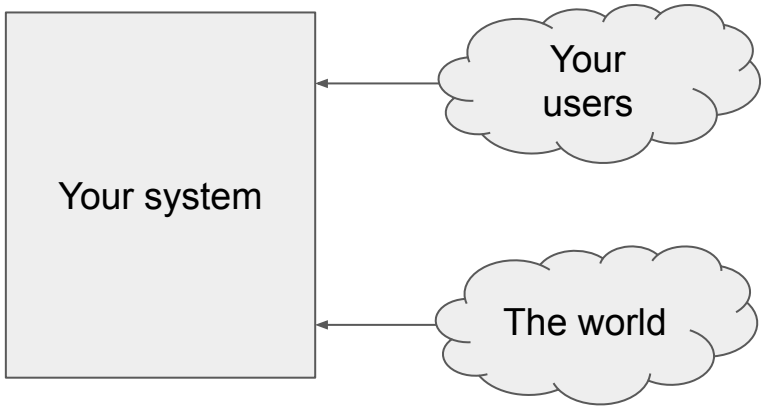
One of my most "wait what" bug

- A pretty tough network-partition occurred on an 70+ Apache Hadoop cluster
- The cluster could not restart
  - There was a NullPointerException during startup caused by its faulty state
  - Has been patched in newer version of HDFS
  - We backported the patch and then distribute the jar
- This is the type of code that feels off from production requirements
  - You hit the bug at the worst moment, during recovery
  - Rolling out an untested patch on a distributed system under 🔥 is an unpleasant experience

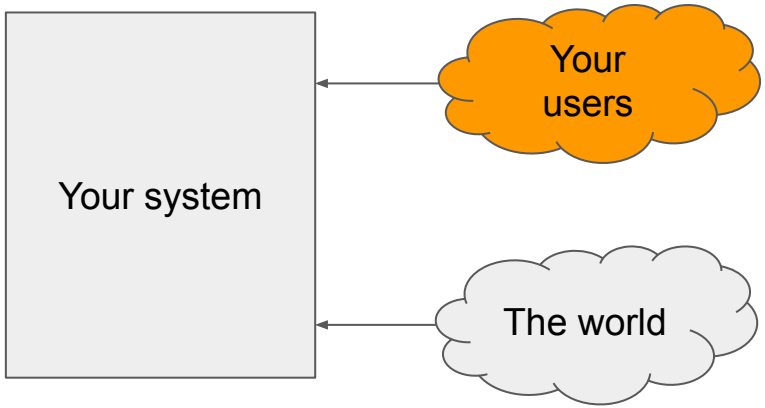
So... What Went Wrong?

- **Prod**uction environments **≠ Dev**elopment environments
  - Like knowledge test vs driving test
- The code didn’t account for real-world complexity like our "not-common" NPE
  - Users will do weird things
  - Systems will be under pressure
  - 💥 And things will break if we’re not ready
- This is a tough work to be paged at 3AM when everything is down, as you need to debug and fix!
  - How can we cultivate a production-oriented culture in juniors without assigning them a pager?

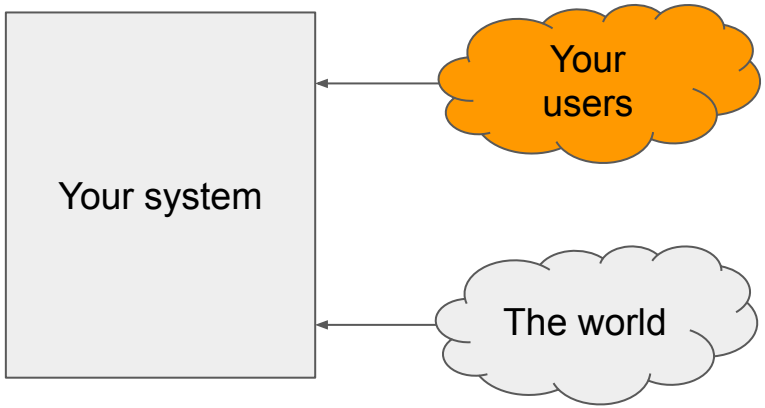
Your system


Your system
Your users
The world

Can we improve user-related code?



Can we improve the **tests** for user-related code?



Let’s test our users

Let’s imagine the following e-commerce API that supports:

- 👤 **User Type**: Guest, Logged-in, Premium
- 💳 **Payment Method**: Credit Card, PayPal, Apple Pay, Gift Card
- 🚚 **Delivery Option**: Standard, Express, In-Store Pickup
- 🎁 **Promotion Applied**: Yes, No
- 📦 **Inventory Status**: In Stock, Low Stock, Out of Stock
- 💱 **Currency**: USD, EUR, GBP

To cover all possibilities, we need to write

3 × 4 × 3 × 2 × 3 × 3 = **648** unique test cases 😱

(just to cover the happy path)

Let’s test our users

Let’s imagine the following e-commerce API that supports:

- 👤 **User Type**: Guest, Logged-in, Premium, **Business**
- 💳 **Payment Method**: Credit Card, PayPal, Apple Pay, Gift Card, **Bank Transfer**
- 🚚 **Delivery Option**: Standard, Express, In-Store Pickup, **Same-Day**
- 🎁 **Promotion Applied**: Yes, No, **Expired Promo**
- 📦 **Inventory Status**: In Stock, Low Stock, Out of Stock, **Preorder**
- 💱 **Currency**: USD, EUR, GBP, **JPY**

To cover all possibilities, we need to write

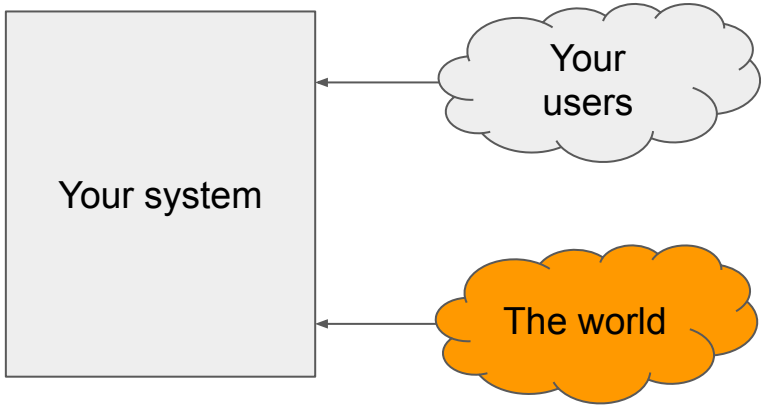
4 × 5 × 4 × 3 × 4 × 4 = **3,840** unique test cases 😱

(just to cover the happy path)

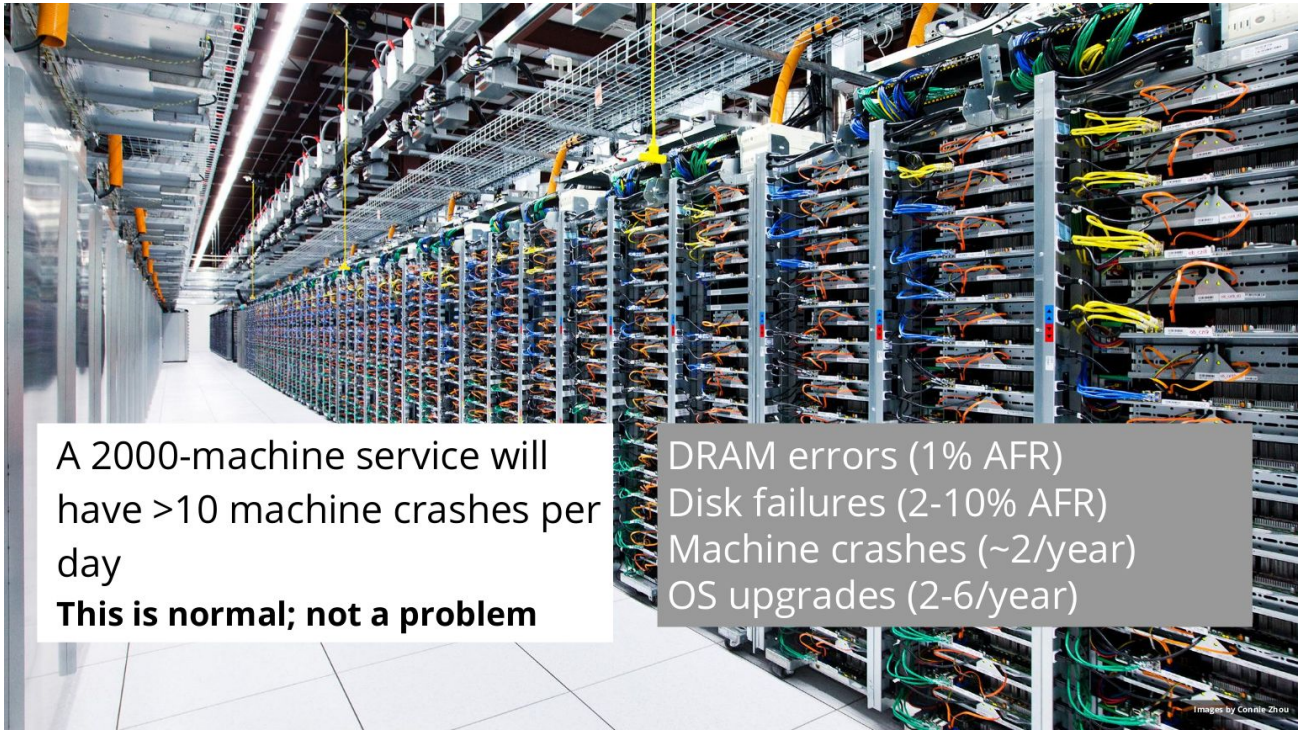
🤯 This Is Why E2E Testing Is Hard

- Every new feature or edge case multiplies the **complexity**
- Manual or naive testing strategies **can’t keep up**
  - Do we need to write a million unit tests on complex systems?
  - Bugs hide in rare, high-dimensional combos
  - You may spend days writing tests with a relative small feature
- We cannot know how our software will be used
- **We cannot test what we don’t know**
  - Tests can't assure bug absence; they mainly check for regressions.

Can our code handle the world?



What is happening in the world?



https://qconlondon.com/london-2015/system/files/keynotes-slides/2015-03%20QCon%20(john%20wilkes).pdf

An Analysis of Network-Partitioning Failures in Cloud Systems(OSDI ’18)

Table 1. List of studied system. The table shows systems’ consistency model, number of failures, and number of catastrophic failures. Highlighted rows indicate systems we tested using NEAT, and the number of failures we found.

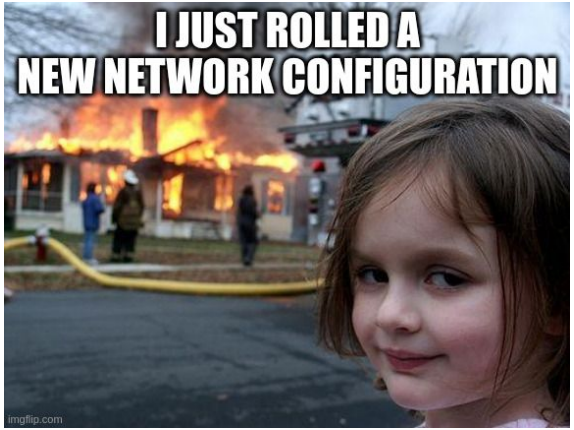
System	Consistency Model	Failures	
		Total	Catastrophic
MongoDB [31]	Strong	19	11
VoltDB [33]	Strong	4	4
RethinkDB [52]	Strong	3	3
HBase [56]	Strong	5	3
Riak [57]	Strong/Eventual	1	1
Cassandra [58]	Strong	4	4
Aerospike [59]	Eventual	3	3
Geode [60]	Strong	2	2
Redis [32]	Eventual	3	2
Hazelcast [29]	Best Effort	7	5
Elasticsearch [28]	Eventual	22	21
ZooKeeper [61]	Strong	3	3
HDFS [1]	Custom	4	2
Kafka [30]	-	5	3
RabbitMQ [62]	-	7	4
MapReduce [4]	-	6	2
Chronos [63]	-	2	1
Mesos [64]	-	4	0

Table 2. The impacts of the failures. The percentage of the failures that cause each impact. Broken locks include double locking, lock corruption, and failure to unlock.

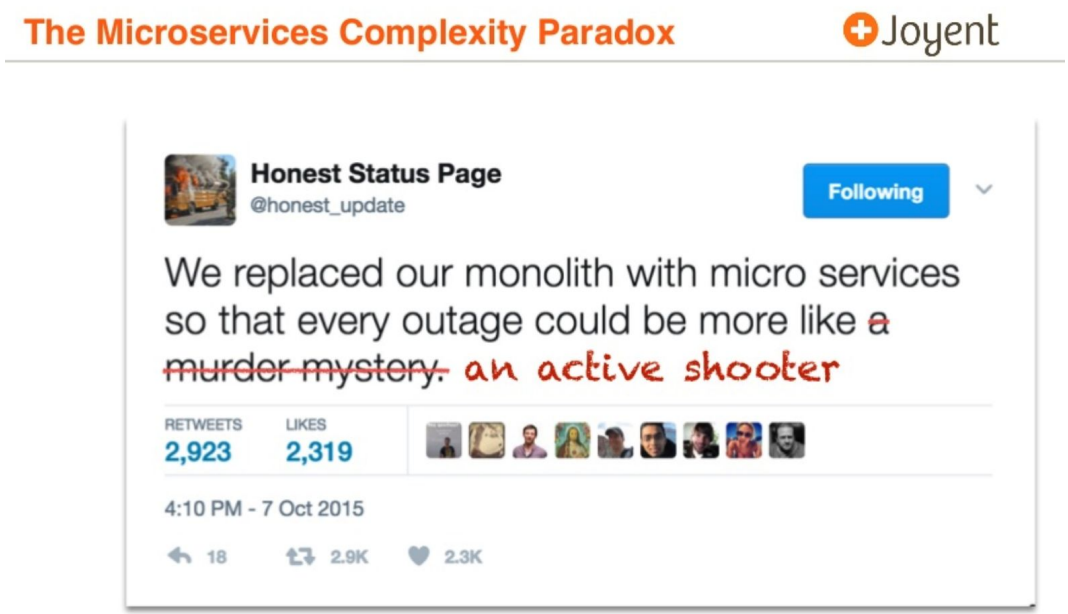
Impact	%	
Data loss	26.6%	Catastrophic (79.5%)
Stale read	13.2%	
Broken locks	8.2%	
System crash/hang	8.1%	
Data unavailability	6.6%	
Reappearance of deleted data	6.6%	
Data corruption	5.1%	
Dirty read	5.1%	
Performance degradation	19.1%	
Other	1.4%	

An Analysis of Network-Partitioning Failures in Cloud Systems(OSDI ’18)

- 80% of the studied failures have **a catastrophic impact**
  - **data loss** being the most common (27%)
  - 90% of the failures are **silent**
- 21% of the failures lead to **permanent damage** to the system
  - This damage persists even after the network partition heals.
- 83% of the failures triggered by a network partition **require an additional** three **events** to manifest

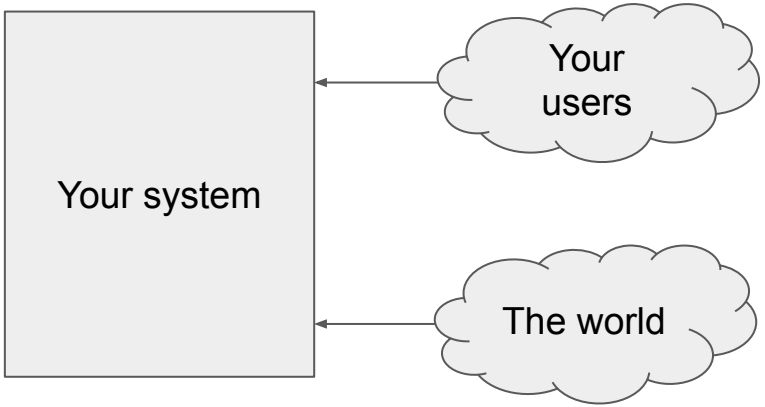


Yes, your micro-services architecture is a distributed system

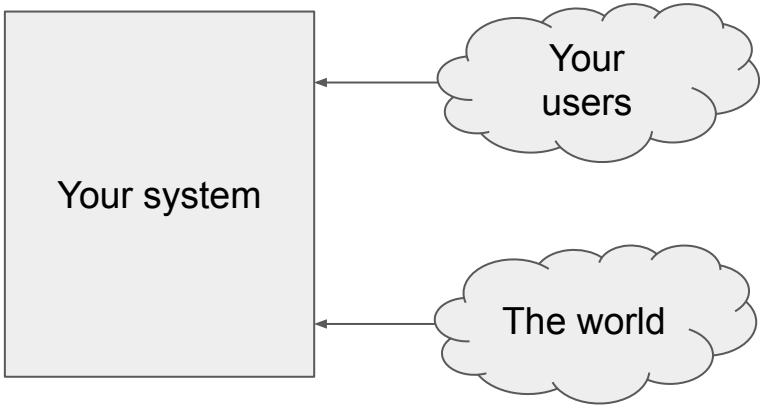


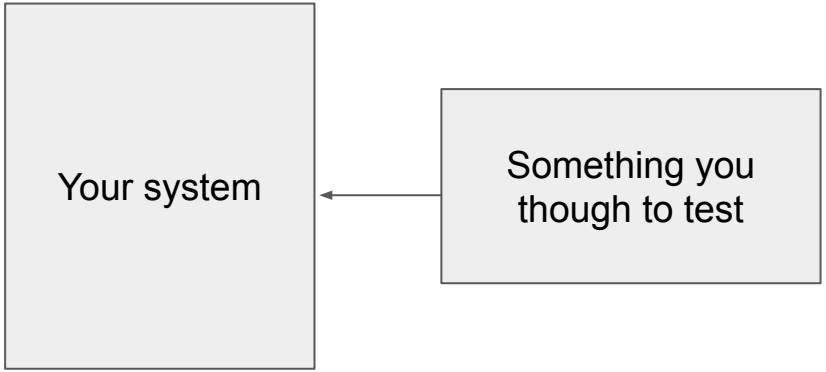
GOTO 2017 • Debugging Under Fire: Keep your Head when Systems have Lost their Mind • Bryan Cantrill

SRE vs SWE



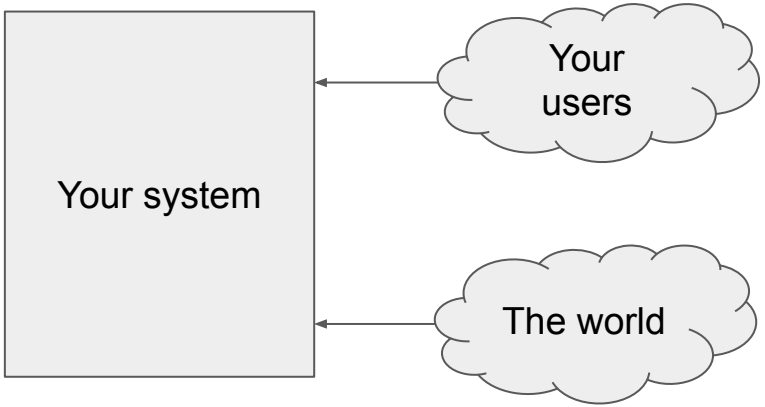
SRE vs SWE

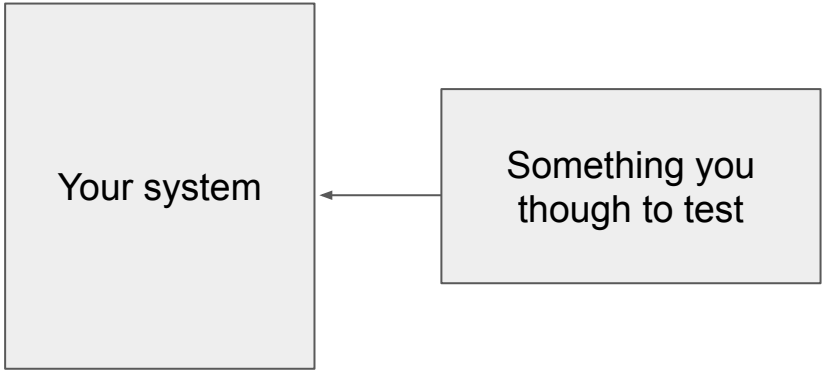

Your system
Your users
The world


Your system
Something you though to test

SRE vs SWE

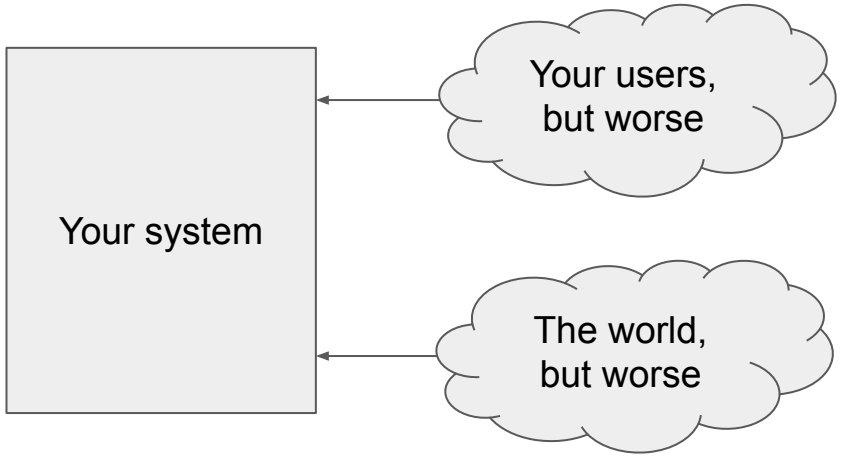
How do we improve this?





We Must test the Worst

- To ensure software behaves correctly
- To surface hidden edge cases
- To uncover the unknown unknowns



Testing needs?

- Fast and debuggable
- Entire system
- Robust

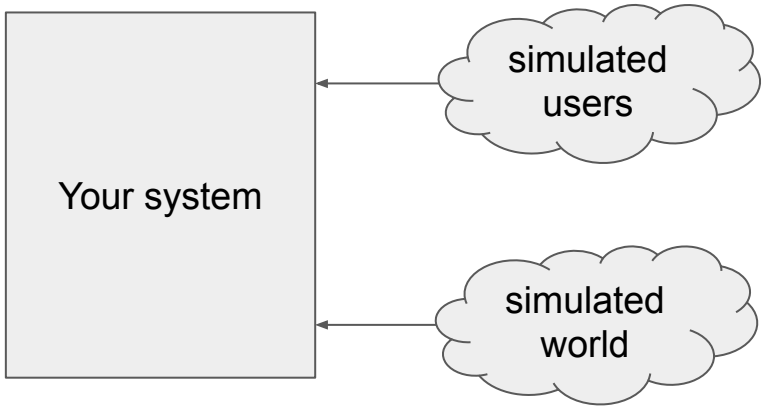
Testing needs?

- Fast and debuggable → Single thread
- Entire system → Single binary
- Robust → Randomized

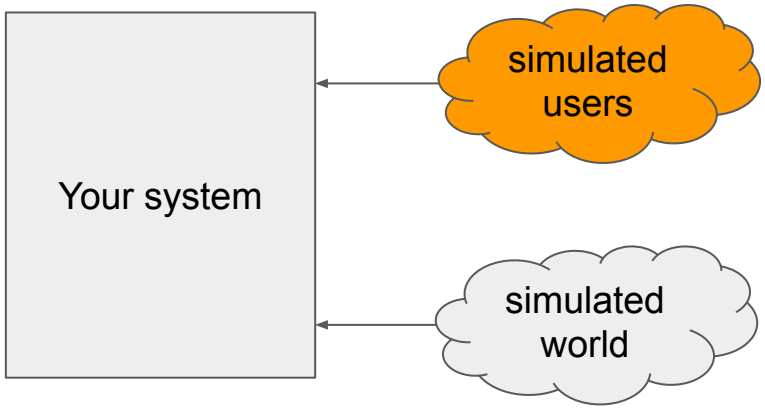
Testing needs?

- Fast and debuggable → Single thread → **Deterministic** Event Loop
- Entire system → Single binary → Network **Simulation**
- Robust → Randomized → **Entropy** Injection

Simulating all the things



How to do simulation-driven development on our users?



Simulating users: randomized input

Write several tests

Write a generator of tests

```
enum UserType { GUEST, LOGGED_IN, PREMIUM, BUSINESS }
enum PaymentMethod { CARD, PAYPAL, APPLE_PAY, GIFT_CARD, BANK_TRANSFER }
// …

Random rand = new Random(); // random seed

UserType user = pickRandom(rand, UserType.values());
PaymentMethod paymentMethod = pickRandom(rand, PaymentMethod.values());
```

Simulating users: checking the state

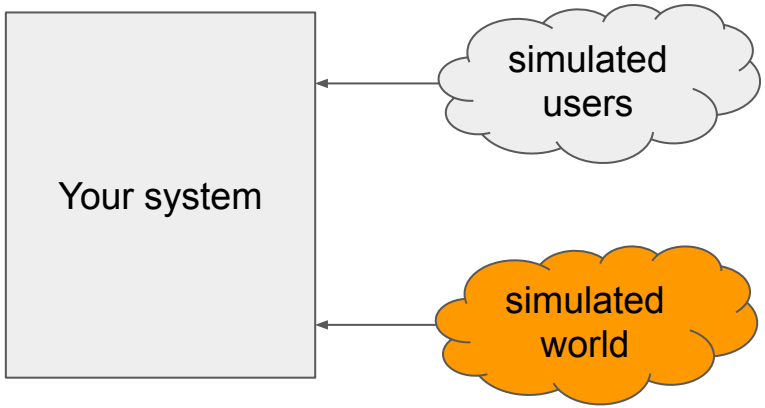
```
// move from hardcoded test case
assertFalse(new User(GUEST).canUse(SAVED_CARD));

// to a test property
assertEquals(user.isAuthenticated(), user.canUse(SAVED_CARD));
```

Yep, it's "just" property-based testing

- Python: https://hypothesis.readthedocs.io/en/latest/
- Java: https://jqwik.net/
- Rust: https://docs.rs/proptest/latest/proptest/

How to do simulation-driven development on the world?



Simulating the World 🌐

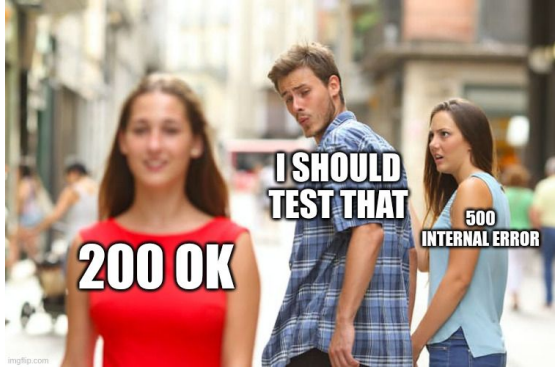
- We need to inject **chaos**😈 on our world 🌍
  - 🕰️ Time (delays, timeouts, retries, race conditions)
  - 🌐 Network (latency, failure, disconnection)
  - 💾 Infrastructure (disk full, service crash, replica lag)
  - ⚙️ External dependencies (slow API, rate limiting)
  - 📊 Load (10 users vs. 10,000 users)
- To inject, you need to **control everything**
  - You need either:
    - To build your systems with DST in mind from day 1
    - Use DST librairies
      - might be called Discrete-event simulation
    - Create mocks that has failures

Simulating the World: the mock-way

```
class HttpClientMock {
    private final Random random = new Random(); // random seed

    String get(String url) {
        // Simulate random chance of returning an error
        if (random.nextDouble() < 0.2) {
            return "HTTP 500 Internal Server Error";
        }

        int delay = random.nextInt(500); // Simulate 0-499ms latency
        Thread.sleep(delay);
        return "HTTP 200 OK";
    }
}
```



Who does deterministic simulation testing?

- **Clever Cloud**
- Antithesis
- TigerBeetle
- Turso
- Resonate
- RisingWave
- Sync @ Dropbox
- sled.rs
- Kafka’s KRaft
- Astradot
- Polar Signals
- AWS

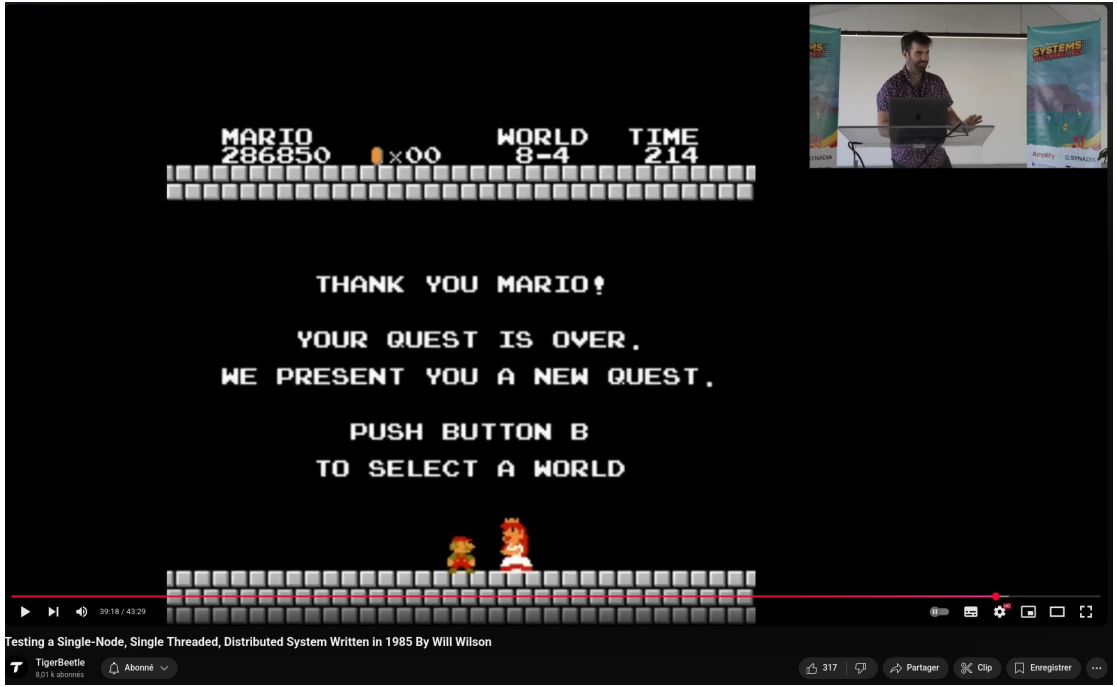
How far can we go in DST?

```python
import mario
import random

INPUT_LENGTH = 80 # FIXME: What should this constant be?

input = random.randbytes(INPUT_LENGTH)

mario.run_input(input)
```



Testing a Single-Node, Single Threaded, Distributed System Written in 1985 By Will Wilson

What we do with DST at Clever Cloud?

We are implementing a **multi-tenant, multi-model distributed database** 🦀🚀

What we do with DST at Clever Cloud?

We are implementing a **multi-tenant, multi-model distributed database** 🦀🚀 thanks to DST ⚙️

```
[[test]]
testTitle = 'Clogged'

    [[test.workload]]
    testName = 'Cycle'
    transactionsPerSecond = 1000.0

    [[test.workload]]
    testName = 'RandomClogging'

    [[test.workload]]
    testName = 'Attrition'
    machinesToKill = 10
    machinesToLeave = 3
```

```
[[test]]
testTitle = 'Clogged'

    [[test.workload]]
    testName = 'Cycle'
    transactionsPerSecond = 1000.0

    [[test.workload]]
    testName = 'RandomClogging'

    [[test.workload]]
    testName = 'Attrition'
    machinesToKill = 10
    machinesToLeave = 3
```

What is trying to achieve

```
[[test]]
testTitle = 'Clogged'

    [[test.workload]]
    testName = 'Cycle'
    transactionsPerSecond = 1000.0

    [[test.workload]]
    testName = 'RandomClogging'

    [[test.workload]]
    testName = 'Attrition'
    machinesToKill = 10
    machinesToLeave = 3
```

Introduce random network partitions

```
[[test]]
testTitle = 'Clogged'

    [[test.workload]]
    testName = 'Cycle'
    transactionsPerSecond = 1000.0

    [[test.workload]]
    testName = 'RandomClogging'

    [[test.workload]]
    testName = 'Attrition'
    machinesToKill = 10
    machinesToLeave = 3
```

Reboot up to 10 machines, keep at least 3 running

```
[[test]]
testTitle = 'Clogged'

    [[test.workload]]
    testName = 'Cycle'
    transactionsPerSecond = 1000.0

    [[test.workload]]
    testName = 'RandomClogging'

    [[test.workload]]
    testName = 'Attrition'
    machinesToKill = 10
    machinesToLeave = 3
```

```
state bool swap = killType == ISimulator::Reboot && BUGGIFY_WITH_PROB(0.75) &&
                    g_simulator->canSwapToMachine(localities.zoneId());
if (swap)
        availableFolders[localities.dcId()].push_back(myFolders);
```

```
[[test]]
testTitle = 'Clogged'

    [[test.workload]]
    testName = 'Cycle'
    transactionsPerSecond = 1000.0

    [[test.workload]]
    testName = 'RandomClogging'

    [[test.workload]]
    testName = 'Attrition'
    machinesToKill = 10
    machinesToLeave = 3
```

Will happen **concurrently**

```
[root@1cd975872551 ~]# fdbserver -r simulation -f /root/logical_db.toml -b on --trace-format json
Random seed is 1976494493...
DeterministicRandom successfully bound to OpenSSL RNG
enabling tss for simulation in mode 1: tss_count:=2 tss_storage_engine:=3
Datacenter 0: 9/9 machines, 1/1 coordinators
```

```
1 tests passed; 0 tests failed.
Waiting for DD to end...
Set perpetual_storage_wiggle=0 ...
Set perpetual_storage_wiggle=0 Done.

Unseed: 46741
Elapsed: 393.556120 simsec, 6.322076 real seconds
```

```
[root@1cd975872551 ~]# fdbserver -r simulation -f /root/logical_db.toml -b on --trace-format json
Random seed is 1856230122...
DeterministicRandom successfully bound to OpenSSL RNG
Datacenter 0: 2/12 machines, 1/1 coordinators
sim http machines = 3
Datacenter 1: 2/12 machines, 0/1 coordinators
sim http machines = 3
Datacenter 2: 2/12 machines, 0/1 coordinators
sim http machines = 2
Datacenter 3: 2/12 machines, 0/1 coordinators
sim http machines = 1
Datacenter 4: 2/12 machines, 0/1 coordinators
sim http machines = 3
Datacenter 5: 2/12 machines, 0/1 coordinators
```

```
1 tests passed; 0 tests failed.
Waiting for DD to end...
Set perpetual_storage_wiggle=0 ...
Set perpetual_storage_wiggle=0 Done.

Unseed: 15957
Elapsed: 357.475100 simsec, 10.749373 real seconds
```

Simulation-driven development

- The simulation is run **in a loop**
  - On the **CI**
  - Continuously in the cloud
- **30 minutes** of simulation = **24 hours** of chaos testing
- When a faulty seed is found, they can **replay it locally**
- It truly feels like **super-powers** because we **trust** our software 🦸

FDB Simulation Bot APP 11:39 PM
Simulation has found a faulty seed 😱
Seed: `1777715850`
Workload: `/root/logical_db.toml`
Commit: `09690a9cdebb89b557e4194b87ab28eb5c925a3f`

DST for Dev (or AI) Education 📚

With deterministic simulation testing:

- ✅ We catch bugs before production
- ✅ We empower devs to build robust and resilient systems
- ✅ We close the gap between dev and prod
- ✅ We provide a feedback-loop for AI 🤔

Even without full DST, you can leverage some **tactics**:

1. Add deterministic randomness
2. Build test properties, not test cases
3. Validate often

Come and visit
Clever Cloud's booth!

Thank you!

Do you have questions?

More links and resources: https://pierrezemb.fr/posts/learn-about-dst/

Please leave your feedback!

Pierre Zemb
https://pierrezemb.fr
@PierreZ